

Design Patterns Refactoring Report (SynChef)

Scope

This report documents pattern-driven refactoring applied to the existing Spring Boot backend of SynChef.

Before vs After

Before

- Notification creation logic was repeated directly in `NotificationService` using manual `setters`.
- Recipe scaling had algorithm logic embedded in a single `service class` (`RecipeScalingService`) with hard-coded timing and rounding behavior.
- `RecipeController` depended directly on multiple preparation-related `services`.

Problems in Original Implementation

- Duplicated construction logic for `AppNotification` increased risk of inconsistent defaults.
- Scaling algorithms were difficult to `swap` or extend without editing core `service` logic.
- Controller-level coupling made orchestration logic less modular and harder to test.

After

- Notification creation is centralized with `NotificationFactory` and entity `Builder`.
- Scaling behavior uses injected strategies for timer scaling and ingredient rounding.
- `RecipePreparationFacade` provides a single orchestration surface for recipe preparation concerns.

Applied Pattern 1: Builder (Creational)

- Pattern Name: Builder
- Applied In: `AppNotification` model
- Files:
 - `backend/src/main/java/edu/cit/batawang/synchef/model/AppNotification.java`
- Justification: Notification entity has many optional fields and flags; builder improves readability and prevents `setter-heavy` construction.
- Improvement:
 - Clearer object construction
 - Safer defaults (`isRead`, `isSystem`) via `@Builder.Default`

Snippet:

```
AppNotification.builder()
    .recipientId(recipientId)
    .type(TYPE_SYSTEM_WELCOME)
    .title(title)
    .message(message)
    .isRead(false)
    .isSystem(true)
    .build();
```

Applied Pattern 2: Factory Method (Creational)

- Pattern Name: Factory Method
- Applied In: Notification creation workflows
- Files:
 - `backend/src/main/java/edu/cit/batawang/synchef/service/notification/NotificationFactory.java`
 - `backend/src/main/java/edu/cit/batawang/synchef/service/NotificationService.java`
- Justification: Object creation was duplicated in `service methods`; factory encapsulates creation rules.
- Improvement:
 - Consistent notification construction
 - Lower duplication
 - Easier extension for new notification types

Applied Pattern 3: Strategy (Behavioral)

- Pattern Name: Strategy
- Applied In: recipe scaling calculations
- Files:
 - backend/src/main/java/edu/cit/batawang/synchef/service/scaling/IngredientRoundingStrategy.java
 - backend/src/main/java/edu/cit/batawang/synchef/service/scaling/MeasurementAwareRoundingStrategy.java
 - backend/src/main/java/edu/cit/batawang/synchef/service/scaling/TimerScalingStrategy.java
 - backend/src/main/java/edu/cit/batawang/synchef/service/scaling/LogarithmicTimerScalingStrategy.java
 - backend/src/main/java/edu/cit/batawang/synchef/service/RecipeScalingService.java
- Justification: Scaling had embedded algorithm decisions; strategy enables replacing algorithm behavior without rewriting core service flow.
- Improvement:
 - Better extensibility (future strategies for regional rounding or custom timing)
 - Better separation of concerns

Snippet:

```
scaledQty = ingredientRoundingStrategy.round(scaledQty, ri.getUnit());  
scaledTimer = timerScalingStrategy.scale(scaledTimer, scalingFactor);
```

Applied Pattern 4: Facade (Structural)

- Pattern Name: Facade
- Applied In: recipe preparation integration point for controllers
- Files:
 - backend/src/main/java/edu/cit/batawang/synchef/service/RecipePreparationFacade.java
 - backend/src/main/java/edu/cit/batawang/synchef/controller/RecipeController.java
- Justification: Controller directly depended on multiple services for related concerns.
- Improvement:
 - Reduced controller coupling
 - Cleaner API boundary for preparation-related use cases

Snippet:

```
return ResponseEntity.ok(recipePreparationFacade.getScaledRecipe(id, servings));
```

Functional Verification

- Backend compile command executed:
 - mvn -DskipTests compile
- Result: BUILD SUCCESS

Final Impact

- Code Organization: improved by isolating creation logic and algorithm logic
- Reusability: improved through factory and strategy abstractions
- Maintainability: improved through reduced duplication and clearer responsibilities
- Scalability: improved with extension points for new notification types and scaling algorithms

Notes for Submission

- Branch created: feature/design-patterns-refactor
- These markdown files can be exported to PDF for submission requirements:
 - docs/DESIGN_PATTERNS_RESEARCH.md
 - docs/DESIGN_PATTERNS_REFACTORING_REPORT.md